# Report 2: RNNs & Transformers

Axel Langenskiöld

May 15, 2026

## 1 Introduction

The topic chosen to train a GPT2 model on was *home renovations*. Home renovation is a domain well-suited to a small language model, since most renovation tasks, like installing a door, patching drywall, refinishing a floor, replacing a faucet, follow a similar structure: prepare the area and gather materials, execute the work (cut, fasten, seal, build) and finish (paint, clean, inspect). The vocabulary is smaller and the task is instructional rather than open-ended. A combination of a narrow lexicon and a repeatable *prepare–build–finish* structure is exactly the kind of pattern a GPT2 scale model can be expected to internalize from a small dataset.

## 2 Related Work - Gathering Data

For this assignment all training data was generated synthetically using a large language model, specifically OpenAI's GPT-5.4. Using synthetic data isn't necessarily the best option, but since we fine-tuned and trained a GPT2 model on specifically *home renovations*, it was optimal since the target domain is narrow enough that a public dataset was hard to find or non-existent. This approach is sometimes referred to as synthetic data generation or LLM-supervised distillation, and it allows a smaller model (GPT-2 architecture) to inherit some of the domain knowledge of a much larger model through fine-tuning.

The chosen domain was home renovations. It's broad enough to allow for some variation between samples (different types of repairs, materials, tools, and skill levels), but still narrow enough that a small model can realistically pick up the vocabulary and structure from only a few hundred examples. To expose the model to multiple styles of supervision, two different output formats were requested from the LLM across separate API calls: a plain text format and an instruction/output format. Each call to the API used one of these templates, sampled at random, which yielded a dataset covering both free-form explanations and structured instructions.

To increase variation in the data, three prompts were used. These where selected at random from `generate_random_prompt(...)`-function.

After generation, the raw model responses were saved under data/raw/ and the rest handled by the prepare_data pipeline, which performed length filtering, duplicate removal, and an 80/20 train/eval split.

## 3 Method

### 3.1 Tokenizer

Instead of training a tokenizer from scratch, the model uses the pre-trained GPT-2 tokenizer from Hugging Face, which is a byte-pair encoding (BPE) tokenizer. BPE works by starting from individual bytes and then repeatedly merging the most common pairs of symbols in a large training corpus into single tokens. After many rounds of merging, the result is a vocabulary made up of common full words, common word fragments, and single characters. When the tokenizer is used, it applies the same merges to a new piece of text, so a frequent word like *paint* becomes a single token, while a rare word like *polyurethane* is broken into a few smaller pieces it has seen before. This results in BPE almost never produces unknown tokens.

**Prompt 1**

I need to generate data to fine tune an LLM on **{*topic*}**. The data should be structured as a JSON array, where each entry has a "text" field, like this:

```
[
  {''text'': ''{Thing
related to {topic}}:
{detailed explanation,
materials, and steps}''},
  {''text'': ''...''}
]
```

I need **{*n_samples*}** samples. Use real, concrete examples drawn from **{*topic*}**; every example must be unique and cover a different sub-topic, technique, or scenario.

**Prompt 2**

Task: produce fine-tuning data for a language model focused on **{*topic*}**.

Requirements:

- Return a JSON array, with a top-level list and no wrapper object.
- Each element is an object with a single key "text".
- Exactly **{*n_samples*}** elements.
- Every "text" value is a self-contained passage about **{*topic*}**, drawing on real, concrete details.
- No two entries may overlap in sub-topic, technique, or scenario.

Shape:

```
[
  {''text'': ''...''},
  {''text'': ''...''}
]
```

**Prompt 3**

I'm fine tuning a GPT-2 style model. I need you to generate data for it. I'm fine tuning the model on **{*topic*}**, so all data should be centered around **{*topic*}**. The structure should be a JSON array, where each entry has "instruction" and "output" fields, like this:

```
[
  {''instruction'':
''{question or task about
{topic}}'',
   ''output'': ''{detailed
answer about {topic}}''},
  {''instruction'': ''...'',
   ''output'': ''...''}
]
```

I need **{*n_samples*}** unique instructions. The returned object must contain only the JSON; no commentary, no markdown fences. Use a wide range of **{*topic*}**-related tasks so every instruction is distinct.

Figure 1: Three prompt formulations used for generating fine-tuning data.

## 3.2 Custom small LM implementation and configuration

The from-scratch model is a small, decoder-only Transformer. Even though the model is only trained on only a few hundred home-renovation samples, the goal was to keep the architecture close enough to a "real" language model that the training loop, tokenizer, and evaluation pipeline could be reused unchanged for the LoRA experiments later on.

Each input token is mapped through a learned token embedding and a learned absolute positional embedding, summed, and passed through a stack of Transformer encoder blocks configured in causal mode (using PyTorch's `nn.TransformerEncoderLayer` with `norm_first=True` and a causal attention mask generated per forward pass). After the final block, a `LayerNorm` is applied and the hidden states are projected back to vocabulary logits by a linear head. The head and token embedding share weights, which roughly halves the parameter count of the input/output layers and tends to train faster on small datasets. The feed-forward dimension inside each block is $4\times$ the embedding size and GELU-activation was used.

The configuration used for training is shown in Table 1.

## 3.3 LoRA fine-tuning approach and configuration

For the second model a pre-trained GPT-2 base model was downloaded from Hugging Face and fine-tuned to the *home-renovation*-dataset using *Low-Rank Adaptation* (LoRA). The idea behind LoRA is that fine-tuning does not really need to move every weight in the model. In practice, the update applied to a large weight matrix during fine-tuning carries less information than the matrix itself, and can be approximated by a much smaller low-rank matrix. LoRA takes advantage of this by freezing the original weights entirely and only learning this small update alongside them.

The base model is a pre-trained GPT-2 (124M parameters). Rather than fine-tuning all weights, the model is wrapped by `peft.get_peft_model`, which inserts a small LoRA adapter at every chosen

| Hyperparameter | Value |
|---|---|
| `n_layer` | 4 |
| `n_head` | 4 |
| `n_embd` | 256 |
| `block_size` | 256 |
| `dropout` | 0.1 |
| Batch size | 16 |
| Optimizer | AdamW, $\beta = (0.9, 0.95)$, wd $= 0.1$ |
| Learning rate | $3 \times 10^{-4}$ |
| Epochs | 3 |

Table 1: Hyperparameters used to train the from-scratch custom LM.

weight matrix and freezes everything else. For each adapted matrix $W$, the adapter adds a pair of trainable low-rank matrices $A$ and $B$. Only these are optimized, while $W$ itself never receives a gradient. Scalar $\alpha/r$ controls how strongly the adapter contributes to the forward pass.

The LoRA adapter is only attached to the `c_attn` module in each transformer block, where `c_attn` is a single linear layer that builds the query, key, and value projections used by self-attention, so changing it is the easiest way to affect how the model moves information between tokens. Every other part of the model stays frozen. Because of this, the number of trainable parameters drops by several orders of magnitude compared to full fine-tune, which makes training faster and keeps the saved checkpoint small since only the adapter weights have to be stored, not the whole model.

The configuration used for the LoRA fine-tune is shown in Table 2.

| Hyperparameter | Value |
|---|---|
| Base model | GPT-2 (124M) |
| `rank` | 8 |
| `lora_alpha` | 16 |
| `lora_dropout` | 0.05 |
| `target_modules` | `["c_attn"]` |
| `bias` | none |
| `task_type` | `CAUSAL_LM` |
| `max_len` | 256 |
| Batch size | 8 |
| Optimizer | AdamW, wd $= 0$ |
| Learning rate | $2 \times 10^{-4}$ |
| Epochs | 3 |

Table 2: Hyperparameters used for the LoRA fine-tune of GPT-2.

### 3.4 Results

To complement the perplexity numbers, each model was given the same three open-ended prompts (*"In summary,"*, *"The key idea is"*, and *"Today I learned"*) and asked to generate a continuation. Perplexity for all models can be see in Table 3.

**Base GPT-2.** The base model produces fluent, grammatical text but with no connection to the home-renovation domain. Prompted with *"In summary"* it generates a passage about *"high-speed optical scanning technology"* for cancer detection and prompted with *"The key idea is"* it discusses the central nervous system. This is expected, since it was trained on general web text and has no particular bias toward the topic.

**Custom LM.** The from-scratch model stays inside the home-renovation domain and reuses the formatting and vocabulary from the training set. For example, prompted with *"In summary,"* it

generates a checklist (*"... Recommended supplies include sandpaper, measuring tape, clamps, saw, wood screws, pencil, wood glue..."*) and even includes the markers (*"### Instruction:"*) into its output. This is consistent with the perplexity result, which indicates that the trainer clearly memorized the structure of the training data rather than learning the topic in a generalizable way, i.e. extremely overfitted. The output is however on-topic and the noun sequences (materials, tools, steps) are likely for words for a renovation task.

**LoRA fine-tune of GPT-2.** The LoRA model sits somewhere in between the two other models. It stays in the home-renovation domain (generating door frames, garage floors, studs, anchors, caulk, drilling) while still using fluent GPT-2-style sentence structure, but it suffers from visible repetition like listing *"screwdriver, screwdriver, screwdriver"* several times in the same paragraph and it occasionally produces mildly contradictory phrasing. This can be expected behaviour the model was only trained on a few hundred samples: the adapter is strong enough to bias the base model toward the domain vocabulary, but it does not fully eliminate the small-data drift.

| Model | Perplexity |
|---|---|
| Base | 98.2023 |
| Scratch | 1.5394 |
| LoRA | 7.1073 |

Table 3: Perplexity scores for the evaluated models.

# 4 Discussion

The results show the trade-off between the two fine-tuning strategies. The from-scratch custom LM gets a very low perplexity on the evaluation set but only because it has effectively memorised the training distribution. The LoRA fine-tune cannot match that perplexity number, but its perplexity is still an order of magnitude better than the base GPT-2's, and its generations remain grammatical and recognisably on-topic, suggesting that it has learned the domain quite well. For a real-world deployment, the LoRA model would be the more useful of the two, despite the higher perplexity.

# 5 Conclusion

A small model fine-tuned on a narrow domain like home renovations could work as a specialized DIY assistant, for example, an offline app that suggests the tools needed for a task or autocompletes step-by-step instructions.

The clearest improvement is the dataset. All samples were produced by a single LLM from only a few prompt templates, which made the data very repetitive and is probably why the from-scratch model overfit and the LoRA model kept repeating words. A better dataset would be both larger and more varied: more samples, more prompt styles, perhaps even mixed with real renovation text from manuals or how-to articles. A second, cheaper improvement is on the LoRA side. Attaching the adapter to more modules than just `c_attn` and raising the rank from 8 to 16 or 32 would give the fine-tune more capacity without changing the base model.